

Lista de Exercícios 03 — Tipos Abstratos de Dados

Disciplina: Estrutura de Dados I

Universidade: Universidade Federal da Paraíba

Discente: Francisco Éricles dos Santos Nascimento

Questão 1

A declaração `typedef struct ponto Ponto;` informa que o tipo existe, mas deixa sua estrutura interna escondida. Dessa forma, quem inclui `ponto.h` utiliza as funções da interface, sem acessar diretamente os campos. Dois exemplos que o compilador recusa são `Ponto p;` (erro esperado: storage size of 'p' isn't known) e `p->x` (erro esperado: invalid use of incomplete typedef 'Ponto').

Questão 2

Considerando os cabeçalhos da biblioteca-padrão necessários para `malloc` e `printf`:

- **(a) Compila.** É possível declarar um ponteiro para um tipo incompleto.
- **(b) Não compila.** O compilador não conhece o tamanho de um objeto `Ponto`.
- **(c) Compila.** `criarPonto` faz parte da interface e devolve `Ponto *`.
- **(d) Não compila.** O campo `x` não está visível em `main.c`.
- **(e) Compila.** A alocação é para dez ponteiros, cujo tamanho é conhecido.
- **(f) Compila.** `sizeof(Ponto *)` consulta o tamanho do ponteiro.
- **(g) Não compila.** `sizeof(Ponto)` precisa da definição completa do tipo.
- **(h) Compila.** Apenas copia o endereço armazenado em `p` para `q`.

Questão 3

(a) A chamada que tenta receber o retorno por valor não funciona no cliente: `main.c` não conhece o tamanho nem a representação de `Ponto`. A assinatura isolada não fornece a definição necessária para trabalhar com o valor retornado.

(b) Seria necessário expor em `ponto.h` o corpo completo de `struct ponto`, com seus campos. Isso eliminaria a opacidade da estrutura e faria o programa cliente depender de sua representação interna.

Questão 4

(a) Considerando a implementação utilizada em aula, na qual `lerPonto(NULL, ...)` retorna 0, a saída é:

5.0

4.0

4.0 6.0
0

(b) Estado imediatamente depois de `a = NULL;`:

Pilha (main)	Heap
-----	-----
<code>a = NULL</code>	ponto criado para <code>a</code> :
<code>b = endereço B</code>	-----> <code>B: {x = 4.0, y = 6.0}</code>
<code>x = 4.0</code>	
<code>y = 6.0</code>	

O ponto que era apontado por `a` (`{4.0, 2.0}`) já foi liberado.

(c) `liberarPonto(a)` recebe `NULL` na segunda chamada; na implementação adotada, isso é tratado sem tentar liberar um bloco novamente. Se `a = NULL` fosse retirada, o ponteiro ficaria pendente e a segunda liberação tentaria liberar a mesma região de memória duas vezes, causando comportamento indefinido.

Questão 5

(a) e (b) Os três problemas são:

1. **Vazamento de memória**, no laço: cada `p` criado deixa de ser acessível ao fim da iteração sem passar por `liberarPonto(p)`.
2. **Liberação indevida de um objeto recebido como parâmetro**, em `liberarPonto(origem)`: a função libera uma memória que pertence ao chamador, sem que esse contrato tenha sido combinado.
3. **Uso após liberação**, no `return`: a expressão `distancia(origem, origem)` utiliza um ponteiro cujo objeto já foi destruído.

(c) Versão corrigida (considerando `origem` válido; em falha de alocação, devolve a soma já obtida):

```
float somaDistancias(Ponto *origem, int n) {  
    float s = 0.0f;  
    for (int i = 0; i < n; i++) {  
        Ponto *p = criarPonto((float)i, (float)i);  
        if (p == NULL) return s;  
        s += distancia(p, origem);  
        liberarPonto(p);  
    }  
    return s; /* distancia(origem, origem) seria zero. */  
}
```

O objeto `origem` continua sob a responsabilidade de quem chamou a função.

Questão 6

(a) Devolver float * para um campo interno expõe a representação de Ponto: se ela mudar, o cliente pode deixar de funcionar. Além disso, esse endereço só é válido enquanto o objeto existir; depois de sua liberação, o ponteiro devolvido fica pendente.

(b) Exemplo de erro em tempo de execução:

```
Ponto *p = criarPonto(2.0f, 3.0f);
float *px = enderecoX(p);
liberarPonto(p);
printf("%f\n", *px); /* acesso a memoria ja liberada */
```

Questão 7

(a) Uma possível interface opaca para Data é:

```
#ifndef DATA_H
#define DATA_H

typedef struct data Data;

/* Cria uma data valida; devolve NULL para data inexistente
   ou se nao houver memoria. O chamador deve libera-la. */
Data *criarData(int dia, int mes, int ano);

/* Libera a data. Aceita NULL. */
void liberarData(Data *d);

/* Copia os tres campos para as variaveis indicadas.
   Devolve 1 no sucesso e 0 para argumentos invalidos. */
int lerData(const Data *d, int *dia, int *mes, int *ano);

/* Coloca -1, 0 ou 1 em resultado (a anterior, igual ou posterior a b).
   Devolve 1 no sucesso e 0 para argumentos invalidos. */
int compararDatas(const Data *a, const Data *b, int *resultado);

/* Avanca um numero nao negativo de dias, cuidando de meses,
   anos e anos bissextos. Retorna 1 no sucesso e 0 em erro;
   em caso de erro, a data original nao deve ser alterada. */
int avancarDias(Data *d, int quantidade);

#endif
```

(b) Em `data.c` ficam a definição de struct `data`, as implementações das operações e as funções auxiliares de validação (dias do mês e ano bissexto). Elas não ficam no cabeçalho para impedir que `main.c` acesse ou modifique diretamente os campos.

(c) A invariante é que todo objeto `Data` existente representa uma data real do calendário, com mês de 1 a 12 e dia compatível com o mês e o ano (incluindo fevereiro em anos bissextos). Isso deve ser conferido na criação e preservado pelo avanço de dias. Leitura, comparação e liberação não modificam a data.

Questão 8

O código está na pasta `questao-08-matriz`, com `matriz.h`, `matriz.c`, `main.c` e `Makefile`.

(a) Escolhi `float *v`: um bloco único para os elementos. Incluindo a estrutura, são duas alocações, os elementos ficam contíguos na memória e a liberação precisa de apenas dois `free`. No modelo `float **v`, haveria alocações separadas para as linhas e a liberação seria feita linha por linha.

(b) e (c) As operações verificam os índices e os ponteiros. Uma tentativa de leitura ou atribuição fora dos limites retorna 0. A criação verifica as alocações e libera a estrutura se a alocação dos dados falhar. O `main.c` testa acessos válidos e inválidos sem acessar campos internos.

(d) Se apenas `matriz.c` mudar, o `Makefile` recompila `matriz.o` e depois refaz o executável. Se `matriz.h` mudar, recompila tanto `matriz.o` quanto `main.o`, pois os dois dependem do cabeçalho, e então refaz o executável.

Questão 9

O código está na pasta `questao-09-complexos`, com interface opaca, implementação, calculadora e `Makefile`.

(a) Escolhi escrever o resultado em um `Complexo *resultado` criado pelo chamador. Assim, a operação não precisa alocar outro objeto: quem criou resultado continua sendo seu dono e chama `liberarComplexo` no final. As funções retornam 1 no sucesso e 0 quando não conseguem executar a operação.

(b) Na divisão, o denominador é $c*c + d*d$. Se ele for zero, a função devolve 0 sem modificar o objeto de resultado, evitando que um erro seja confundido com um número complexo válido.

(c) O `main.c` lê dois números complexos e um operador, chama a função adequada e mostra o resultado usando `lerComplexo`. Ele não precisa conhecer os campos internos de struct complexo.

Questão 10

O código está na pasta `questao-10-vetor-dinamico`, com as duas políticas de crescimento selecionáveis na interface. O contador considera, em cada expansão, quantos elementos antigos

precisariam ser transferidos se realloc precisasse mover o bloco; a linguagem C não permite observar quantas cópias físicas a biblioteca realizou internamente.

(a) e (b) Partindo de capacidade 1 e inserindo 100.000 valores, o modelo de contagem apresenta:

- Crescimento de uma posição: **4.999.950.000** cópias potenciais.
- Crescimento por duplicação: **131.071** cópias potenciais.
- Razão (+1 / duplicação): **38.146,88**.

(c) `inserirVetorD` usa um ponteiro temporário para receber `realloc`. Se o resultado for `NULL`, devolve 0 e não altera o ponteiro, tamanho, capacidade ou conteúdo antigos do vetor.

(d) A duplicação não torna cada inserção mais cara porque os aumentos de capacidade ficam cada vez mais espaçados: algumas inserções exigem a transferência de muitos elementos, mas a maior parte apenas grava um valor na posição livre; o custo médio amortizado de inserir é constante.